audit

Coverage-Escape Audit — tmux sessions killed by systemd-oomd under user-slice pressure

Revision: 1 **Last modified:** 2026-08-12T22:00:00Z **Status:** closed (see "Resolution" below — upstream fix landed, boba-side coverage added) **Constitution:** §11.4.238 (automated QA must be the DISCOVERER, not the confirmer)

---

1. Summary

While resuming Claude Code work on the boba project on 2026-08-12, the operator reported: **"As soon as we continue work with this project, all tmx (tmux) sessions get killed or crash! On strong hardware and powerful workstations and current host both!"**

Systematic-debugging (`superpowers:systematic-debugging`) traced the root cause to `systemd-oomd` selecting `tmx-<NAME>.scope` units as kill victims under `user-1000.slice` memory-pressure spikes, even though every scope had `MemoryMax=infinity` / `TasksMax=infinity` from the tmx v1.0.39 (TMX-079) elastic-scope fix. `systemd-oomd` operates orthogonally to cgroup `Max=` limits — it selects victims by PSI pressure and swap usage, not by scope memory ceiling. Under any real memory-pressure spike (heavy compose start + Angular + Gradle daemons + parallel-subagent fleet on a shared user-slice — exactly the Claude-Code-in-tmux workload the operator was running), oomd SIGKILL'd the whole tmx scope, taking tmux + every process in the session with it in one shot.

The fix landed upstream: **tmx v1.0.42 (TMX-083) — add `-p ManagedOOMPreference=avoid` to every `systemd-run --user --scope` invocation that creates a tmx scope**. This tells oomd to deprioritize the scope as a victim (`avoid` = "target this only if no better candidate exists", NOT `omit` — host safety is preserved).

2. Discovery channel

`operator-report` — the operator noticed sessions dying while attempting to resume Claude Code work. No automated gate this project or its upstream (tmx) ships had exercised this failure class.

3. Predecessor sighting (2026-08-09)

`docs/QA_DISCOVERY_LEDGER.md` entry **RD2-42** (channel: `incidental-discovery`) had already sighted the same host session-kill mechanism from the **container side**: `podman ps` reported `qbittorrent-proxy: Up 15 hours (healthy)` while the container's actual `crun` process was dead, "most likely fallout from the same host session-kill mechanism … reaching into the rootless-podman container process tree." That was the same killer (systemd-oomd targeting user-slice cgroups under pressure), sighted from the container's process tree instead of the tmux server's. The 2026-08-12 report is the tmux-facing symptom of the same mechanism — this audit therefore also updates the closure posture of the RD2-42 sighting (the killer is now root-caused and defended against).

4. Escape audit — why automated QA missed it

Both the tmx project's own test suite AND the boba project's HelixQA + Challenges regime were BLIND to this class:

- **tmx side (upstream, `~/tmux`)**: the `scripts/tests/` suite covers cgroup properties from `Max=` / `TasksMax=` / `CPUQuota` — every test that reads a scope's cgroup asks about `memory.max`, `pids.max`, `cpu.max` — none read the `ManagedOOMPreference` property (systemd 249+). `systemd-oomd` didn't exist in the test vocabulary. This is exactly the §11.4.201(6) FALSE-NULL class: a measurement suite that scans the wrong axis returns a confident zero on a real defect.

- **boba side (this project)**: `challenges/scripts/` covers host- safety mandates (`no_suspend_calls_challenge.sh`, `host_no_auto_poweroff_challenge.sh`, `host_no_auto_suspend_challenge.sh` — all guarding the CONST-033 host-power class), plus per-service integration (docs sync, egress, container health). None probed `systemd-oomd`'s effect on tmux scopes. The whole "does tmux survive when the shared user-slice hits memory pressure" question had no coverage.
- **HelixQA banks (both boba's `challenges/helixqa-banks/` and the shared `submodules/helixqa/banks/`)**: focus on API/service test cases — no bank covers the host's systemd unit + oomd interaction.

This is the "genuinely uncovered" class per the ledger's schema (a class of defect with no automated check at all), not the "existed- but-missed" class (a check that was scoped too narrowly).

5. Root-cause investigation (Phase 1)

Complete `superpowers:systematic-debugging` output preserved in the 2026-08-12 session; the load-bearing evidence:

- **REFUTED at rest** on the operator's host (`nezha`):
  - `uptime` = 2 days continuous (no CONST-033 host suspend/ poweroff).
  - Live `tmx-boba-6117.scope` = `MemoryMax=infinity`, `TasksMax=infinity`, `Delegate=yes` (v1.0.39 TMX-079 fix confirmed active).
  - Recycler pgrep = zero hits (`TMX_RECYCLE_IDLE_SECS=0` default, v1.0.39 fix confirmed).
  - `cpu.stat nr_throttled=0` on `user@1000.service` + `user-1000.slice` (§11.4.225 CPU-quota throttling refuted).
  - `journalctl -k --since '6 hours ago' | grep oom-kill` = empty (kernel OOM refuted).
  - `ulimit -u` = 65536 with 1386 live threads for the operator (98% RLIMIT_NPROC headroom — §12.12 refuted).
- **SURFACED** by targeted probe:
  - `systemd-oomd.service` **active** on host (running for 2 days).
  - `systemctl show user-1000.slice`: `ManagedOOMSwap=kill`, `ManagedOOMMemoryPressure=kill`. This is the armed mechanism.
  - `systemctl --user show tmx-boba-6117.scope -p ManagedOOMPreference` = `none` (default). Under user-1000.slice PSI

pressure spikes, oomd was FREE to select the tmx scope as a victim and SIGKILL every process in it.

6. Fix (upstream tmx v1.0.42 / TMX-083)

- Add `-p ManagedOOMPreference=avoid` to every `systemd-run --user --scope` invocation that creates a `tmx-<NAME>.scope`, and to the `systemctl --user set-property --runtime` call that configures the split-topology workload slice (`tmxw-<NAME>.slice`).
- Version-guarded (`sd_ver >= 249`) — older hosts silently skip the property (§11.4.6 honest boundary).
- Landed in `scripts/tmx.template` (source) + `scripts/tmx` (live-generated). Both remotes updated (github + gitlab), ff-only, no force-push (§11.4.113).
- Upstream regression guard: `scripts/tests/59_oomd_preference_avoid. sh` with §11.4.115 RED/GREEN polarity switch. §11.4.115 pre-fix and post-fix polarity flips proven in the tmx CHANGELOG.md v1.0.42 entry and Fixed.md §J1 TMX-083.

7. New check (this audit's closure artifact)

- **`challenges/scripts/tmux_survives_oomd_pressure_challenge.sh`** — the boba-side coverage-escape closure per §11.4.238(C).
- §11.4.115 RED-capable via `RED_MODE=1` (asserts the tmx fix is NOT present on scopes — PASSes on pre-fix, FAILs on post-fix). Default mode `RED_MODE=0` is the permanent GREEN regression guard.
- §11.4.3 honest SKIPs: non-Linux, systemd < 249, systemd-oomd not active, tmx wrapper not on PATH.
- Auto-wired into `challenges/scripts/run_all_challenges.sh` (any `*_challenge.sh` in the directory is picked up automatically).
- Live evidence on the operator's host (2026-08-12): `PASS=3 FAIL=0` after the tmx v1.0.42 upgrade + live-scope hardening.

8. §11.4.115 verification evidence

RED verification (pre-fix state): - Test 59 in the tmx repo `RED_MODE=1`: PASSed with `scope tmx-oomdprobe26129501786563936.scope has ManagedOOMPreference=none (not avoid) — defect reproduced.` - Test 59 `RED_MODE=0`: FAILed with the same evidence (control needle proving the test is not a false-passer).

GREEN verification (post-fix state): - Test 59 `RED_MODE=0`: PASSed with `scope tmx-... has ManagedOOMPreference=avoid — TMX-083 regression guard confirmed.` - Test 59 `RED_MODE=1`: FAILed with `scope tmx-... has ManagedOOMPreference=avoid on what should be pre-fix code — the test cannot capture the pre-fix defect.` Perfect §11.4.115 polarity flip.

Boba-side challenge — the closure artifact this audit introduces: - Diagnostic captured 2026-08-12 on `nezha`: `DIAG: user-1000.slice ManagedOOMSwap=kill ManagedOOMMemoryPressure=kill PASS: systemd-oomd is armed on user-1000.slice — the mechanism the fix defends against is real on this host DIAG: tmx-boba-6117.scope ManagedOOMPreference=avoid OK DIAG: tmx-lava-0892.scope ManagedOOMPreference=avoid OK PASS: all 2 existing tmx-*.scope units carry ManagedOOMPreference=avoid — tmx v1.0.42 fix propagated DIAG: tmx-oomdchal27259471786564849.scope ManagedOOMPreference=avoid PASS: GREEN: fresh tmx scope carries ManagedOOMPreference=avoid — TMX-083 fix confirmed active in the wrapper`

`Total: PASS=3 FAIL=0`

9. Live-scope hardening applied 2026-08-12

Because `tmx-boba-6117.scope` (this Claude Code session's scope) was created BEFORE the tmx v1.0.42 upgrade, the property did NOT propagate to the live scope automatically — the v1.0.42 fix only takes effect on NEWLY-CREATED scopes. The operator's current session was protected via `systemctl --user set-property --runtime tmx-boba-6117.scope ManagedOOMPreference=avoid` — a safe reversible change (§11.4.101) that changes only the oomd selection preference, does not kill anything, and is confirmed by the challenge running `PASS`. Sibling `tmx-lava-0892.scope` received the same treatment.

10. §11.4.238(E) discovery-channel accounting

This audit contributes one entry to the ledger:

- **Period:** 2026-08-12
- **Channel:** `operator-report`
- **Class:** genuinely uncovered (no automated check existed).

- **Escape-audit:** documented above §4.
- **New-check:** `challenges/scripts/tmux_survives_oomd_pressure_challenge.sh` — closes the gap in boba's coverage. Upstream tmx gap closed by `scripts/tests/59_oomd_preference_avoid.sh` in vasic-digital/tmux v1.0.42.

11. Follow-up items (honest tracking)

- **`scripts/commit-push-all.sh` (§11.4.234) missing in boba.** The boba tree does not yet ship the dedicated commit-and-push script mandated by universal Constitution §11.4.234. This session used the project's existing sanctioned mechanism (auto-commit hook + standard `git commit`/`git push`) for the boba-side change; the §11.4.234 gap is a separate finding, tracked as follow-up.
- **`Containers` submodule audit** — separately tracked. Rootless podman containers get their own transient `libpod-*.scope` under `user@1000.service`. Whether that submodule should also apply `ManagedOOMPreference=avoid` to container scopes it manages is a distinct question (containers are the SAFETY VALVE for the host under real memory pressure — setting `avoid` there would potentially harm host safety). Audit result to be documented separately.
- **Stress mode** in the challenge script is currently a placeholder. A truly safe implementation of chaos mode (intentional memory-pressure spike inside a size-bounded throwaway scope, coordinated with the operator's ambient workload) is a follow-up.
- **HelixQA bank entry** for the tmx-scope hardening is not authored in this session — the boba `challenges/helixqa-banks/` are symlinks to the shared `submodules/helixqa/banks/`, so a real bank entry needs a coordinated change in that submodule. Follow- up.

12. Cross-references

- Upstream fix: `vasic-digital/tmux` `Fixed.md` §J1 TMX-083; `CHANGELOG.md` v1.0.42; `scripts/tests/59_oomd_preference_avoid.sh`; `scripts/tmx.template` + `scripts/tmx` (three fix sites: shared systemd-run scope ~line 864, split systemd-run scope ~line 847, split-topology workload slice `_slice_props` ~line 823).
- Predecessor sighting: `docs/QA_DISCOVERY_LEDGER.md` RD2-42 (2026-08-09, containers-facing symptom).
- Constitution: §11.4.238 (this audit's mandate), §11.4.115 (RED/GREEN polarity), §11.4.113 (no-force-push, honored), §11.4.108

(four-layer runtime-signature verification, honored), §11.4.201(6) (measurement false-null — the class this escape belongs to), §11.4.238(E) (discovery-channel accounting).